TCP-Lab 讲义0：整体思路

> 你最终要做的事：在实验提供的"模拟网络/转发服务器"上，让 `TCP_Sender` 把应用层数据可靠地交付到 `TCP_Receiver`，即使信道会**出错/丢包/延迟**（组合由 TCP 首部 `eFlag` 控制）。

---

0. 项目里都有什么（你真正要改哪里）

实验材料分成三类：

1. **你要实现的工程（重点）**
   `TCP_Test_1.zip` 解压后是 Java 工程，需要改：

- `src/com/ouc/tcp/test/TCP_Sender.java`
- `src/com/ouc/tcp/test/TCP_Receiver.java`
- `src/com/ouc/tcp/test/CheckSum.java`

2. **参考源码（理解系统用）**
   `TCP_TestCode/`：包含发送/接收监听线程、计时器封装、报文结构等（用来理解"平台怎么工作"）。
3. **调试工具（可选但强烈建议）**
   `OUC-TCP-Lab-Visualizer/`：把运行产生的 Log 可视化，帮助你定位"丢的是数据还是 ACK、超时是否按 3s 触发、窗口是否右移"等。

---

1. 推荐实现顺序（从最小闭环到最终 TCP Reno）

强烈建议按"能跑 → 能抗出错 → 能抗丢包 → 能扩展窗口 → 再做拥塞控制"的顺序迭代，每一步都有可验证的现象。

Step 1：跑通最小闭环（不抗差错）

- 目标：Sender 能发、Receiver 能收、Receiver 能回 ACK、能生成 `recvData.txt`。
- 只在 `eFlag=0`（无差错）下验证。
- 你会看到 Log 全部 ACKed。

Step 2：RDT2.0（只解决"位错/出错"）

- 目标：数据包"被改坏"时，Receiver 能识别并拒收（或回 -1/重复 ACK），Sender 能据此重传。
- 核心实现：`CheckSum.computeChkSum()`。
- 验证：`eFlag=1`（只出错）下，Log 中会出现出错重传，但 `recvData.txt` 不应出现错误值或重复交付。

Step 3：RDT2.2（停止等待 + 只有 ACK）

- 目标：不用 NAK，也能在"出错 ACK / 重复 ACK"情况下逼迫 Sender 重传当前分组。
- 核心：**序号** + **ACK 携带确认对象** + **重复 ACK 等价于 NAK**。
- 验证：`eFlag=1` 下仍正确；并观察 Sender 是否"只重传当前包"。

Step 4：RDT3.0（停止等待 + 超时重传，解决丢包/延迟）

- 目标：当数据或 ACK 丢了，Sender 在约 3s 超时后重传；延迟导致的重复到达不能造成重复交付。
- 核心：**一个计时器**（Stop-and-Wait 一般够用一个），超时触发重传“当前未确认包”。
- 验证：`eFlag=2/3/6/7`（含丢包/延迟组合）下仍能最终交付完整、无重复。

Step 5：GBN（Go-Back-N，滑动窗口 + 累计确认）

- 目标：窗口内可以连续发多个包；ACK 采用累计确认；超时重传窗口内“未确认的一段”（常见实现：从 base 起把窗口内全部重传）。
- 关键约束（常见评分点）：**发送方只用 1 个计时器。**
- 验证：`eFlag` 各种情况下：
  - 出错/丢包时，是否“整段回退重传”（而不是只重传一个包）。
  - Receiver 是否**不缓存失序包**（GBN 接收方通常直接丢弃失序）。

Step 6：SR（Selective Repeat，选择重传）

- 目标：窗口内按包确认；只有丢/错的包需要重传；Receiver 需要缓存失序包并按序交付。
- 核心：每包独立计时（或用更复杂的结构模拟），Receiver 缓存 + 连续段交付。
- 验证：在丢包/延迟时，重传是否“精确命中丢的那几个包”；最终 `recvData.txt` 是否无乱序交付。

Step 7：TCP（在本实验平台语义下的“更像 TCP 的版本”）

通常会在 GBN/SR 的基础上加入：

- Receiver：**延迟确认（约 500ms）** + **累计确认**（正常传输）
- 出错时：放弃累计确认，改发**重复 ACK**（为 Reno 的快重传铺路）
- Sender：超时重传窗口内“所有未确认包”（注意不要重传已确认的包）
- 仍尽量满足：发送方 **1 个计时器**

Step 8：拥塞控制（Tahoe → Reno）

在 TCP 的滑动窗口之上叠加“拥塞窗口 cwnd”逻辑：

- Tahoe：慢开始、拥塞避免（加法增大）、超时后乘法减小/回慢开始
- Reno：在 Tahoe 基础上加：3 个重复 ACK → 快重传；快恢复

---

2. 平台“长什么样”（先把系统想象清楚）

你写的 Sender/Receiver 并不直接连真实 TCP，而是发自定义对象 `TCP_PACKET` 到一个**模拟 Server**：



关键点：

- `ListenACK` / `ListenPacket` 是后台线程：收到了就回调你写的 `recv()` / `rdt_recv()`。

- `eFlag` 决定信道注入的异常类型（0~7 的组合）。
- 评测通常看：Log + `recvData.txt` + 你代码是否符合要点（计时器数量、窗口移动、确认策略等）。

2.1 `eFlag` 速查表（PPT 中给的 0~7）

通常含义如下（不同版本平台描述略有差异，但基本一致）：

eFlag	信道异常类型
0	无差错
1	只出错（位错）
2	只丢包
3	只延迟
4	出错 / 丢包
5	出错 / 延迟
6	丢包 / 延迟
7	出错 / 丢包 / 延迟

> 实操建议：每做完一档协议，就固定在 `eFlag=0` 先验证功能，再切到你需要覆盖的异常组合验证鲁棒性。

2.2 术语速查（写代码前先统一“名词”）

- `seq`（序号）：你给每个数据包的编号（可以是“包序号”，也可以是“字节流序号”）。实验通常不强制哪一种，但 Sender/Receiver 必须一致。
- `ack`（确认号）：确认到哪个序号（同样要和你的 `seq` 语义匹配）。
- Stop-and-Wait：一次只允许 1 个未确认数据包在路上。
- `base`：发送窗口里“最早未确认”的序号。
- `nextSeq`：下一个要发送的序号。
- `N`：窗口大小（GBN/SR/TCP 都有）。
- `rwnd`：接收窗口（接收端能接收/缓存的能力）。
- `cwnd`：拥塞窗口（网络层面的“我敢发多少”）。